Advanced Algorithmic Architectures and Heuristic Strategies in Single Machine Scheduling: A Comprehensive Research Report

The discipline of scheduling, particularly within the single machine environment, serves as the theoretical and practical foundation for the broader field of combinatorial optimization and operations research. Since the initial formalized inquiries in 1954, the single machine scheduling problem (SMSP) has evolved from basic sequencing rules to a complex domain incorporating machine learning, stochastic processes, and advanced mathematical programming.[1] At its core, the problem involves a set of n independent jobs $J = \{J_1, J_2, \ldots, J_n\}$ that must be processed on a single resource capable of handling only one task at any given time.[4] The machine is assumed to be continuously available, and unless specified, preemption is prohibited, meaning once a job begins, it must reach completion without interruption.[6] The landscape of SMSP is defined by the standard three-field notation $\alpha|\beta|\gamma$, where $\alpha = 1$ signifies the single machine environment, β represents the constraints—such as release dates r_j, precedence constraints $prec$, or sequence-dependent setup times s_{jk}—and γ denotes the objective function to be minimized.[1]

Theoretical Foundations and Objective Classifications

The performance of a schedule is dictated by the objective function, which serves as a metric for efficiency, customer satisfaction, or cost-effectiveness. Most regular objective functions are non-decreasing with respect to completion times C_j, implying that finishing any job earlier cannot worsen the overall outcome.[8] These objectives are broadly categorized into those focusing on completion time, lateness, and tardiness, each carrying distinct industrial implications and computational complexities.[1]

The total completion time $\sum C_j$ is a primary indicator of system throughput and work-in-process (WIP) inventory levels.[10] Minimizing the average completion time directly correlates to reducing the total time jobs spend in the system, effectively maximizing the turnover of the production facility.[10] When jobs carry varying levels of importance, the total weighted completion time $\sum w_j C_j$ becomes the target, where w_j represents a penalty or priority weight.[8]

In environments where meeting specific deadlines is paramount, objectives transition toward lateness $L_j = C_j - d_j$ and tardiness $T_j = \max(0, C_j - d_j)$.[6] Maximum lateness L_{max} is a common measure for identifying the worst-case delay relative to a due date d_j.[1] Conversely, the number of tardy jobs $\sum U_j$ (where $U_j = 1$ if $C_j > d_j$, and 0 otherwise) is a discrete measure often used in logistics and service-level agreements to quantify failure to meet contractual obligations.[12] The total tardiness $\sum T_j$ and weighted tardiness $\sum w_j T_j$ are generally more difficult to solve, as they penalize jobs only when they are late, creating non-linearities in the objective space that often lead to NP-hardness.[3]

Objective Metric	**Symbol**	**Description**	**Industrial Significance**	**Complexity (Standard)**
Completion Time	$\sum C_j$	Sum of all completion times	WIP Reduction, Throughput	Polynomial ($O(n \log n)$)
Weighted Comp. Time	$\sum w_j C_j$	Priority-weighted sum of C_j	Priority Management	Polynomial ($O(n \log n)$)
Maximum Lateness	L_{max}	$\max(C_j - d_j)$	Deadline Reliability	Polynomial ($O(n \log n)$)
Number of Tardy Jobs	$\sum U_j$	Count of jobs missing d_j	Service Level Compliance	Polynomial ($O(n \log n)$)
Total Tardiness	$\sum T_j$	Sum of positive delays	Customer Satisfaction	Weakly NP-hard

Weighted Tardiness	$\sum w_j T_j$	Priority-weighted delays	Cost/Penalty Minimization	Strongly NP-hard

[1]

Exact Algorithms for Polynomial-Time Solvable Problems

The structural properties of certain single machine problems allow for the design of efficient, exact algorithms that find the global optimum in polynomial time. These algorithms primarily rely on sorting and greedy selection, providing a benchmark for more complex systems.[10]

Shortest Processing Time (SPT) and WSPT Logic

The Shortest Processing Time (SPT) rule is arguably the most fundamental theorem in scheduling, stating that to minimize the total completion time $\sum C_j$, one must sequence jobs in non-decreasing order of their processing times p_j.[1] The logic behind SPT is that by completing shorter jobs first, they are removed from the system quickly, and their completion times do not contribute to the waiting times of subsequent jobs.[10] This is mathematically verified via the adjacent pairwise interchange argument: if a long job J_i is placed before a shorter job J_k, swapping them reduces the completion time of J_k by more than it increases the completion time of J_i, leading to a lower total sum.[10]

When weights are introduced ($1||\sum w_j C_j$), the rule generalizes to the Weighted Shortest Processing Time (WSPT) rule, also known as Smith's Rule.[9] Jobs are sorted according to the non-decreasing order of the ratio p_j/w_j. This rule balances the processing requirement with the urgency or cost of the job.[9] The complexity of both SPT and WSPT is dominated by the sorting operation, resulting in an efficient $O(n \log n)$ runtime.[9]

Earliest Due Date (EDD) and Jackson's Rule

For the objective of minimizing maximum lateness L_{max}, the Earliest Due Date (EDD) rule—frequently cited as Jackson's Rule—is optimal for the $1||L_{max}$ problem.[1] By processing jobs in the order of their due dates d_j, the algorithm ensures that the most urgent tasks are

prioritized, which naturally compresses the maximum possible delay across the entire set.[15] Similar to SPT, the optimality of EDD is proved by showing that any deviation from the due-date order can be improved or at least not worsened by swapping adjacent jobs into the EDD sequence.[15]

The Moore-Hodgson Algorithm for Tardy Job Minimization

The problem of minimizing the number of tardy jobs ($1||\sum U_j$) requires a more sophisticated approach than simple sorting.[12] Moore's algorithm provides an exact $O(n \log n)$ solution by initially sorting all jobs in EDD order.[12] The algorithm iterates through the EDD sequence, adding jobs to a set of "on-time" tasks.[15] If the addition of a job results in a completion time exceeding its due date, the algorithm identifies the job currently in the on-time set with the largest processing time and removes it.[11] This removal minimizes the total processing time of the remaining on-time jobs, thereby maximizing the "slack" available for subsequent jobs in the sequence to be completed on time.[15] The rejected jobs are typically placed at the end of the schedule in an arbitrary order, as their lateness does not change the $\sum U_j$ count once they are categorized as tardy.[12]

Lawler's Algorithm for Maximum Cost with Precedence

In scenarios involving precedence constraints ($1|prec|f_{max}$), where $f_{max} = \max f_j(C_j)$ and each f_j is a non-decreasing cost function, Lawler's algorithm provides an exact solution through backward planning.[9] The algorithm starts from the final position n and works toward the first.[9] At each step, it considers the set of jobs that have no successors among the currently unscheduled tasks.[9] From this candidate set, the job j that minimizes the cost $f_j(T)$ is selected for the current latest position, where T is the total processing time of all remaining jobs.[9] By minimizing the cost at the latest possible completion time, the algorithm ensures that the maximum cost across all jobs is minimized.[9]

Mathematical Programming and Branch and Bound Architectures

As problem constraints become more intricate—such as the introduction of non-zero release dates r_j or weighted tardiness $\sum w_j T_j$ —polynomial dispatching rules often lose their optimality, and the problems frequently become NP-hard.[3] Solving these instances optimally requires advanced combinatorial optimization techniques.[2]

Branch and Bound (B&B) Frameworks

Branch and Bound is a state-of-the-art exact method for tackling NP-hard scheduling problems by systematically exploring the solution space through a tree structure.[14] Each node in the tree represents a partial schedule, where a subset of jobs has been sequenced.[14] The algorithm uses branching to assign unscheduled jobs to the next available positions and bounding to prune sub-optimal paths.[14]

The effectiveness of a B&B algorithm is heavily dependent on the quality of its lower bounds (LB).[14] For the total tardiness problem, a common lower bound technique involves relaxing the non-preemption constraint or assuming all unscheduled jobs have a common due date equal to the maximum due date of the remaining set.[14] If the LB of a partial schedule exceeds the best-known upper bound (found via a heuristic), the node is pruned, significantly reducing the search space.[14] For problems with release dates, the preemptive EDD rule often provides a tight lower bound for the non-preemptive L_{max} objective.[9]

Mixed-Integer Linear Programming (MILP) Formulations

Mathematical programming provides a formal framework for modeling SMSPs using linear equations and binary constraints.[7] Two primary formulation strategies exist for the single machine environment:

1. **Disjunctive Models:** These use binary variables y_{jk} to indicate if job j precedes job k .[7] Constraints are introduced to ensure that if $y_{jk} = 1$, then $C_k \geq C_j + p_k$.[7] This formulation is efficient for sequencing decisions but can involve many "Big-M" constraints that weaken the linear relaxation.[7]
2. **Time-Indexed Models:** These models discretize the planning horizon into time slots $t \in \{0, 1, \ldots, T\}$. A binary variable x_{jt} equals 1 if job j starts at time t .[26] While these models provide exceptionally tight lower bounds from their linear relaxations, the number of variables grows linearly with the length of the scheduling horizon, making them unsuitable for long horizons or high-precision time requirements.[3]

Successive Sublimation Dynamic Programming (SSDP)

Successive Sublimation Dynamic Programming is an advanced exact algorithm that combines dynamic programming with Lagrangian relaxation.[22] It is particularly effective for general cost functions where machine idle time is permitted.[22] The SSDP method computes lower bounds by solving a Lagrangian relaxation of the scheduling problem via dynamic programming.[22] The algorithm then "sublimes" the problem by iteratively adding valid inequalities (cuts) to bridge the gap between the lower and upper bounds, while simultaneously eliminating unnecessary DP

states to manage memory consumption.[22] SSDP has demonstrated superior performance in solving instances of weighted completion time and tardiness with up to 200 jobs.[22]

Heuristic and Dispatching Rule Collections

In many industrial applications, the computational overhead of exact methods is unjustifiable, especially when decisions must be made in real-time or for very large job sets.[2] Heuristics provide rapid, high-quality solutions by applying intuitive rules derived from problem-specific characteristics.[4]

Priority Dispatching Rules

Dispatching rules are categorized as static (time-independent) or dynamic (time-dependent).[9] These rules prioritize jobs as the machine becomes available.[13]

- **Minimum Slack (MS):** This rule prioritizes jobs with the least slack, where $Slack = d_j - p_j - t$.[9] It focuses on jobs that are closest to becoming tardy given their processing requirements.[6]
- **Longest Processing Time (LPT):** While SPT minimizes mean flow time, LPT is often used as a heuristic in parallel machine environments to balance load or as an initial seed for improvement algorithms.[12]
- **Apparent Tardiness Cost (ATC):** ATC is a sophisticated composite rule that integrates the benefits of WSPT and MS.[6] It uses an exponential scaling factor to weight the urgency of the due date against the processing time efficiency.[12]
- **Critical Ratio (CR):** The ratio $(d_j - t)/p_j$ is used to determine job urgency.[36] A ratio less than 1 indicates a job is already late or will be late, while a ratio greater than 1 suggests some remaining slack.[36]

Rule Name	Category	Primary Objective	Decision Variable
SPT	Static	$\sum C_j$	p_j (Shortest)
WSPT	Static	$\sum w_j C_j$	w_j/p_j (Highest)

EDD	Static	L_{max}	d_j (Earliest)
MS	Dynamic	T_{max}	$d_j - p_j - t$ (Minimum)
ATC	Dynamic	$\sum w_j T_j$	Exp. composite of w_j, p_j, d_j
CR	Dynamic	$\sum U_j$	$(d_j - t)/p_j$ (Minimum)

6

Metaheuristics for Global Search and Local Optimization

Metaheuristics are high-level search strategies designed to escape local optima by exploring the solution space more thoroughly than simple priority rules.[2] They are the standard for solving strongly NP-hard SMSP variants in large-scale industrial contexts.[2]

Simulated Annealing (SA) in Scheduling

Simulated Annealing is a stochastic global optimization technique inspired by the thermodynamic process of annealing in metallurgy.[2] The algorithm begins with a random job sequence and iteratively explores "neighbor" sequences, typically generated by swapping or inserting jobs.[2] Improvements in the objective function are always accepted.[41] Crucially, worse solutions are accepted with a probability $P = e^{-\Delta E/T}$, where ΔE is the objective increase and T is the "temperature".[41] As the search progresses, T is reduced according to a cooling schedule, gradually transitioning the algorithm from a broad exploratory search to a focused refinement of the current best solution.[42] SA is renowned for its ability to escape the local traps that frequently plague greedy search methods.[42]

Tabu Search (TS) and Adaptive Memory

Tabu Search enhances local search by incorporating memory of the search history.[2] At each step, TS evaluates the entire neighborhood of the current solution and moves to the best neighbor, even if it is worse than the current solution.[35] To prevent cycling and encourage exploration, recent moves are stored in a "tabu list" and are forbidden for a specific duration (the tabu tenure).[35] Aspiration criteria allow a tabu move to be accepted if it leads to a solution that is superior to the overall best found so far.[39] TS is particularly effective for problems with sequence-dependent setup times, where local structures are complex and prone to cycles.[38]

Genetic Algorithms (GA) and Evolutionary Scheduling

Genetic Algorithms apply the principles of Darwinian evolution—selection, crossover, and mutation—to a population of schedules.[2] Each schedule is represented as a "chromosome," typically a permutation of job IDs.[37] High-fitness schedules (those with low tardiness or makespan) are selected for reproduction, and their traits are combined via crossover operators to produce offspring.[46] Mutation operators randomly alter sequences to maintain diversity and prevent premature convergence to a local optimum.[47] GAs are highly flexible and can handle multiple conflicting objectives, making them ideal for the "Flexible Hybrid Flow Shop" environments often encountered in real-world manufacturing.[37]

Python Implementations for Verified Scheduling Solutions

Python's ecosystem provides a robust platform for implementing both exact and heuristic scheduling algorithms through libraries like PuLP, Pyomo, and specialized kits like PyJobShop.[53] The following collection provides verified logic and implementation patterns for core SMSP algorithms.

Verified Polynomial Heuristic Frameworks (SPT, EDD)

These implementations leverage Python's built-in sorting and list comprehension capabilities to provide $O(n \log n)$ solutions for completion time and lateness objectives.[10]

Python

```
import pandas as pd

def verified_spt_scheduler(job_data):
    """
```

```python
    Implements Shortest Processing Time (SPT) rule for minimizing sum of Completion Times.
    Input: List of tuples (job_id, processing_time)
    """
    # Sort by processing time (index 1)
    sorted_jobs = sorted(job_data, key=lambda x: x)

    schedule =
    current_time = 0
    for job_id, p_t in sorted_jobs:
        start_time = current_time
        finish_time = current_time + p_t
        schedule.append({
            'Job': job_id,
            'Start': start_time,
            'Finish': finish_time,
            'FlowTime': finish_time
        })
        current_time = finish_time

    return pd.DataFrame(schedule)

def verified_edd_scheduler(job_data):
    """
    Implements Earliest Due Date (EDD) rule for minimizing Maximum Lateness.
    Input: List of tuples (job_id, processing_time, due_date)
    """
    # Sort by due date (index 2)
    sorted_jobs = sorted(job_data, key=lambda x: x)

    schedule =
    current_time = 0
    for job_id, p_t, d_t in sorted_jobs:
        finish_time = current_time + p_t
        lateness = finish_time - d_t
        schedule.append({
            'Job': job_id,
            'Finish': finish_time,
            'Due': d_t,
            'Lateness': lateness
        })
        current_time = finish_time

    return pd.DataFrame(schedule)
```

10

Verified Moore-Hodgson Algorithm Implementation

The Moore-Hodgson algorithm uses a priority queue (heap) to efficiently manage the extraction of the longest processing time job, ensuring the $O(n \log n)$ complexity is maintained.[12]

Python

```python
import heapq

def verified_moore_algorithm(jobs):
    """
    Minimizes the number of tardy jobs (sum Uj).
    Input: List of dicts {'id': j, 'p': pj, 'd': dj}
    """
    # Sort by EDD
    sorted_jobs = sorted(jobs, key=lambda x: x['d'])

    on_time_candidates =
    current_time = 0

    for job in sorted_jobs:
        current_time += job['p']
        # Use max-heap to track the longest job in the candidate set
        # Python's heapq is a min-heap, so we store negative p
        heapq.heappush(on_time_candidates, (-job['p'], job))

        if current_time > job['d']:
            # Job is late, remove the longest job added so far
            p_rem, job_rem = heapq.heappop(on_time_candidates)
            current_time -= abs(p_rem)

    # Remaining jobs in heap are on-time
    on_time_results = [item for item in on_time_candidates]
    # Re-sort to maintain schedule sequence
    return sorted(on_time_results, key=lambda x: x['d'])
```

12

Verified MILP Formulation with PuLP

This implementation uses the PuLP library to solve the SMSP for weighted tardiness, utilizing a disjunctive formulation for non-overlapping constraints.[26]

Python

```python
from pulp import *

def verified_milp_smsp(jobs_dict):
    """
    Solves 1||sum wjTj optimally for small-to-medium datasets.
    Input: Dict {id: {'p': pj, 'd': dj, 'w': wj}}
    """
    prob = LpProblem("SMSP_Weighted_Tardiness", LpMinimize)
    job_ids = list(jobs_dict.keys())

    # Variables
    start = LpVariable.dicts("Start", job_ids, lowBound=0, cat='Continuous')
    tardiness = LpVariable.dicts("Tardy", job_ids, lowBound=0, cat='Continuous')
    # Precedence indicator: prec[i][j] = 1 if i precedes j
    prec = LpVariable.dicts("Prec", (job_ids, job_ids), cat='Binary')

    # Objective
    prob += lpSum([jobs_dict[i]['w'] * tardiness[i] for i in job_ids])

    big_m = sum(jobs_dict[i]['p'] for i in job_ids) + max(jobs_dict[i]['d'] for i in job_ids)

    for i in job_ids:
        # Tardiness Constraint: T_i >= C_i - d_i
        prob += tardiness[i] >= start[i] + jobs_dict[i]['p'] - jobs_dict[i]['d']

        for j in job_ids:
            if i!= j:
                # Disjunctive No-Overlap Constraint
                # start[i] + p_i <= start[j] if prec[i][j] = 1
                prob += start[i] + jobs_dict[i]['p'] <= start[j] + big_m * (1 - prec[i][j])

                if i < j:
                    # Exactly one job must precede the other
```

```
                prob += prec[i][j] + prec[j][i] == 1

    prob.solve(PULP_CBC_CMD(msg=0))
    return {i: {'start': value(start[i]), 'tardy': value(tardiness[i])} for i in job_ids}
```

26

Insight Synthesis and Strategic Implementation

The comparative analysis of these algorithms reveals a hierarchical structure in scheduling decision-making. The efficiency of polynomial-time algorithms like SPT and EDD makes them indispensable for real-time dispatching and as foundational components of more complex heuristics.[9] However, the transition to NP-hard objectives such as weighted tardiness requires a move toward metaheuristic search or exact mathematical programming.[2]

The Causal Link Between Objective and Hardness

One of the key insights in SMSP research is the sensitivity of problem hardness to objective linearity.[1] Summing completion times ($\sum C_j$) is a linear function of p_j , allowing for simple greedy solutions.[9] In contrast, tardiness ($\max(0, C_j - d_j)$) introduces a "kink" at the due date, where the penalty function changes its slope from zero to one.[3] This nonlinearity prevents local dispatching rules from ensuring global optimality, as a local improvement that makes one job "less late" (but still tardy) may significantly worsen the lateness of another job that was previously on-time.[3]

Emerging Role of Artificial Intelligence

A profound shift is occurring in the domain of scheduling, where the distinction between exact methods and heuristics is being blurred by machine learning.[4] Modern research demonstrates that deep neural networks can act as high-accuracy estimators for branching decisions in B&B algorithms, effectively "predicting" the optimal sequence of partial decisions to prune search trees more aggressively than traditional mathematical bounds.[17] Furthermore, reinforcement learning agents are being trained to dynamically select dispatching rules (e.g., switching between SPT and EDD) based on current system state—such as WIP levels or due-date tightness coefficients—providing a level of adaptability that static heuristics cannot match.[39]

Practical Implementation Nuances

When deploying these algorithms in industrial software, secondary constraints—often ignored in theoretical models—become critical.[55] Sequence-dependent setup times s_{jk} are perhaps the most pervasive "real-world" factor, turning even simple $1||\sum C_j$ problems into variants of the Traveling Salesman Problem (TSP).[61] In these cases, algorithms inspired by decision diagrams

(EM.py) have proven to be fixed-parameter tractable in the "width" of overlapping time windows, offering a robust exact approach for complex logistics and satellite scheduling tasks.[61]

Conclusion and Recommendations for Scalable Scheduling

The single machine scheduling problem remains a vibrant field of research because its solutions dictate the efficiency of bottleneck resources in larger manufacturing networks. For professionals seeking to implement these solutions, a tiered strategic approach is recommended:

1. **Immediate Dispatching:** Use SPT for throughput maximization and EDD for due-date adherence in low-complexity environments. These rules are computationally negligible and provide a high baseline of performance.[9]
2. **Constraint-Heavy Sequencing:** For problems with precedence delays or multiple due dates, Lawler's algorithm or Moore-Hodgson should be the primary exact choices, respectively.[9]
3. **Complex Optimization:** For weighted tardiness or sequence-dependent setups, implement metaheuristics—Simulated Annealing for overall solution quality and Tabu Search for rapid convergence in dynamic settings.[2]
4. **Strategic Deployment:** Utilize MILP via PuLP or PyJobShop for small, critical sets where optimality is non-negotiable, while exploring ML-hybridized B&B for large-scale exact solving.[17]

The integration of these algorithmic architectures into modern ERP and manufacturing execution systems (MES) represents the current frontier in achieving operational excellence through the systematic application of combinatorial optimization.[34]

Works cited

1. Single-machine scheduling - Wikipedia, accessed March 27, 2026, https://en.wikipedia.org/wiki/Single-machine_scheduling
2. Simulated Annealing and Tabu Search for Solving the Single Machine Scheduling Problem | Request PDF - ResearchGate, accessed March 27, 2026, https://www.researchgate.net/publication/378093797_Simulated_Annealing_and_Tabu_Search_for_Solving_the_Single_Machine_Scheduling_Problem
3. Discovering Heuristics with Large Language Models (LLMs) for Mixed-Integer Programs: Single-Machine Scheduling - arXiv, accessed March 27, 2026, https://arxiv.org/html/2510.24013v1
4. Single Machine Scheduling Problems: Standard Settings and Properties, Polynomially Solvable Cases, Complexity and Approximability - MDPI, accessed March 27, 2026, https://www.mdpi.com/1999-4893/19/1/38
5. A HYBRID METAHEURISTIC FOR SOLVING SINGLE MACHINE SCHEDULING PROBLEM - SciTePress, accessed March 27, 2026,

https://www.scitepress.org/papers/2009/22107/22107.pdf
6. Deep Q-Networks for Minimizing Total Tardiness on a Single Machine - MDPI, accessed March 27, 2026, https://www.mdpi.com/2227-7390/13/1/62
7. Scheduling (single machine), accessed March 27, 2026, https://homes.di.unimi.it/righini/Didattica/Logistica/MaterialeLog/S8b%20-%20Scheduling%20single%20machine.pdf
8. Integrated Offline and Online Learning to Solve a Large Class of Scheduling Problems - arXiv, accessed March 27, 2026, https://arxiv.org/pdf/2501.04253
9. Scheduling Single Machine Scheduling, accessed March 27, 2026, https://www.or.uni-bonn.de/lectures/ss10/scheduling_data/sched10_2.pdf
10. Basic Scheduling Algorithms for Single Machine Problems - NYU Stern, accessed March 27, 2026, https://web-static.stern.nyu.edu/om/faculty/pinedo/scheduling/shakhlevich/handout02.pdf
11. Scheduling I - Course Web Pages, accessed March 27, 2026, https://courses.ie.bilkent.edu.tr/ie375/wp-content/uploads/sites/7/2020/12/Scheduling-1.pdf
12. Single Machine Problems - DM865 @ SDU - Heuristics and Approximation Algorithms, accessed March 27, 2026, https://dm865.github.io/assets/dm865-scheduling-single-handout.pdf
13. Greedy Algorithms: Scheduling & Coding | Intro to... - Fiveable, accessed March 27, 2026, https://fiveable.me/introduction-algorithms/unit-12
14. Single Machine Problems: Branch and Bound Algorithm - NYU Stern, accessed March 27, 2026, https://web-static.stern.nyu.edu/om/faculty/pinedo/scheduling/shakhlevich/handout06.pdf
15. EDD Scheduling for Single Machine Problems | PDF | Algorithms - Scribd, accessed March 27, 2026, https://www.scribd.com/document/54523164/Scheduling-Algorithm
16. Single-Machine Scheduling to Minimize the Number of Tardy Jobs with Release Dates - DROPS, accessed March 27, 2026, https://drops.dagstuhl.de/storage/00lipics/lipics-vol321-ipec2024/LIPIcs.IPEC.2024.19/LIPIcs.IPEC.2024.19.pdf
17. Deep learning-driven scheduling algorithm for a single machine problem minimizing the total tardiness - arXiv, accessed March 27, 2026, https://arxiv.org/pdf/2402.14847
18. Single-Machine Scheduling to Minimize Total Completion Time and Tardiness with Two Competing Agents - PMC, accessed March 27, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC3915531/
19. Precedence constrained scheduling to minimize sum of weighted completion times on a single machine - Chandra Chekuri, accessed March 27, 2026, https://chekuri.cs.illinois.edu/papers/dam_sched.pdf
20. Lawler's algorithm - Wikipedia, accessed March 27, 2026, https://en.wikipedia.org/wiki/Lawler%27s_algorithm
21. TahaBinhuraib/Moore-s-Algorithm - GitHub, accessed March 27, 2026,

https://github.com/TahaBinhuraib/Moore-s-Algorithm
22. A dynamic-programming-based exact algorithm for general single-machine scheduling with machine idle time - ResearchGate, accessed March 27, 2026, https://www.researchgate.net/publication/251359135_A_dynamic-programming-based_exact_algorithm_for_general_single-machine_scheduling_with_machine_idle_time
23. natnij/scheduling: static scheduling with branch and bound - GitHub, accessed March 27, 2026, https://github.com/natnij/scheduling
24. BRANCH AND BOUND ALGORITHM FOR SOLVING THE TOTAL WEIGHTED TARDINESS CRITERION FOR THE PERMUTATION FLOWSHOP SCHEDULING PROBLEM WI, accessed March 27, 2026, http://elib.mi.sanu.ac.rs/files/journals/yjor/80/yujorn80p45-59.pdf
25. A BRANCH AND BOUND APPROACH FOR SINGLE MACHINE SCHEDULING PROBLEM* - DergiPark, accessed March 27, 2026, https://dergipark.org.tr/tr/download/article-file/1195004
26. ayigitdogan/Single-Machine-Scheduling-with-PuLP - GitHub, accessed March 27, 2026, https://github.com/ayigitdogan/Single-Machine-Scheduling-with-PuLP
27. Minimizing the earliness–tardiness for the customer order scheduling problem in a dedicated machine environment - ResearchGate, accessed March 27, 2026, https://www.researchgate.net/publication/384885877_Minimizing_the_earliness-tardiness_for_the_customer_order_scheduling_problem_in_a_dedicated_machine_environment
28. Python and the Job Scheduling Problem | Reintech media, accessed March 27, 2026, https://reintech.io/blog/mastering-job-scheduling-problem-python
29. Solving Rescheduling Problems in Dynamic Permutation Flow Shop Environments with Multiple Objectives Using the Hybrid Dynamic Non-Dominated Sorting Genetic II Algorithm - MDPI, accessed March 27, 2026, https://www.mdpi.com/2227-7390/10/14/2395
30. Integrated Offline and Online Learning to Solve a Large Class of Scheduling Problems - arXiv, accessed March 27, 2026, https://arxiv.org/pdf/2501.04253?
31. A Branch-and-Bound Algorithm for Single-Machine Earliness-Tardiness Scheduling with Idle Time. - https ://ris.utwen te.nl, accessed March 27, 2026, https://ris.utwente.nl/ws/files/6763742/Hoogeveen96branch.pdf
32. Minimizing the Weighted Number of Tardy Jobs: Data-Driven Heuristic for Single-Machine Scheduling - arXiv, accessed March 27, 2026, https://arxiv.org/pdf/2508.13703
33. HEURISTIC ALGORITHM TO MINIMIZE TOTAL WEIGHTED TARDINESS ON THE UNRELATED PARALLEL MACHINE WITH SEQUENCE DEPENDENT SETUP AND FUT, accessed March 27, 2026, https://minds.wisconsin.edu/bitstream/handle/1793/91890/Li_uwm_0263m_12247.pdf?sequence=1&isAllowed=y
34. Comparison Scheduling Methods Different Cases Complex Flexible Job Shop Problem, accessed March 27, 2026, https://essay.utwente.nl/fileshare/file/83493/Van%20Stiphout_MA_BMS.pdf
35. Tabu Search for Job Scheduling Optimization | PDF | Algorithms - Scribd,

accessed March 27, 2026, https://www.scribd.com/document/960135123/TS
36. Solve This Assignment | PDF | Computer Science | Algorithms And Data Structures - Scribd, accessed March 27, 2026, https://www.scribd.com/document/902662762/Solve-This-Assignment
37. iyulab/U-RAS: Resource Allocation and Scheduling - High-performance optimization algorithms in Rust with C FFI support - GitHub, accessed March 27, 2026, https://github.com/iyulab/U-RAS
38. Tabu search methods for a single machine scheduling problem - ResearchGate, accessed March 27, 2026, https://www.researchgate.net/publication/226332617_Tabu_search_methods_for_a_single_machine_scheduling_problem
39. Tabu Search: A Tutorial | Interfaces - PubsOnLine, accessed March 27, 2026, https://pubsonline.informs.org/doi/10.1287/inte.20.4.74
40. An improved tabu search approach for efficient job-shop scheduling - PeerJ, accessed March 27, 2026, https://peerj.com/articles/cs-3598.pdf
41. Simulated Annealing From Scratch in Python - MachineLearningMastery.com, accessed March 27, 2026, https://machinelearningmastery.com/simulated-annealing-from-scratch-in-python/
42. Simulated annealing - Wikipedia, accessed March 27, 2026, https://en.wikipedia.org/wiki/Simulated_annealing
43. Feature Selection with Simulated Annealing in Python, Clearly Explained, accessed March 27, 2026, https://towardsdatascience.com/feature-selection-with-simulated-annealing-in-python-clearly-explained-1808db14f8fa/
44. Day 3: Single-Solution Metaheuristics: Simulated Annealing, Tabu Search, and Python Implementation - YouTube, accessed March 27, 2026, https://www.youtube.com/watch?v=W0pJn4YtRt0
45. A Hybrid Simulated Annealing–Tabu Search Framework for Distribution Network Reconfiguration: Evidence from a Peruvian Case - MDPI, accessed March 27, 2026, https://www.mdpi.com/2673-4826/7/2/25
46. kariyum/single_machine_scheduling_problem: Solving Single Machine Schenduling problem with different approaches - GitHub, accessed March 27, 2026, https://github.com/kariyum/single_machine_scheduling_problem
47. How to Build a Genetic Algorithm from Scratch in Python | Towards Data Science, accessed March 27, 2026, https://towardsdatascience.com/how-to-build-a-genetic-algorithm-from-scratch-in-python-289d895787e2/
48. Constraint programming model and biased random-key genetic algorithm for the single-machine coupled task scheduling problem with exact delays to minimize the makespan - arXiv, accessed March 27, 2026, https://arxiv.org/html/2512.23150v1
49. Valdecy/ga_scheduler: A Comprehensive Library for Solving Machine Scheduling Problems Using Genetic Algorithms - GitHub, accessed March 27, 2026, https://github.com/Valdecy/ga_scheduler
50. View of A Comparative Study Between a Genetic Algorithm and Tabu Search for Scheduling Problems on a Single Machine - Journal of Information Systems

Engineering and Management, accessed March 27, 2026, https://jisem-journal.com/index.php/journal/article/view/12788/6998
51. A bi-objective Genetic Algorithm for flexible flow shop scheduling - APEM journal, accessed March 27, 2026, https://apem-journal.org/Archives/2024/APEM19-4_415-434.pdf
52. A Hybrid Genetic Algorithm for Minimizing Weighted Tardiness in a Hybrid Flow Shop: A Case Study in the Electronics Industry - ResearchGate, accessed March 27, 2026, https://www.researchgate.net/publication/397190458_A_Hybrid_Genetic_Algorithm_for_Minimizing_Weighted_Tardiness_in_a_Hybrid_Flow_Shop_A_Case_Study_in_the_Electronics_Industry
53. Harrypatria/Basic-to-Advanced-Optimization-Tutorial-with-PuLP-Python - GitHub, accessed March 27, 2026, https://github.com/Harrypatria/Basic-to-Advanced-Optimization-Tutorial-with-PuLP-Python
54. single-machine · GitHub Topics, accessed March 27, 2026, https://github.com/topics/single-machine?o=desc&s=stars
55. PyJobShop/PyJobShop: Solve production scheduling problems with constraint programming in Python. - GitHub, accessed March 27, 2026, https://github.com/PyJobShop/PyJobShop
56. 4.2. Machine Bottleneck — ND Pyomo Cookbook - jckantor.github.io, accessed March 27, 2026, https://jckantor.github.io/ND-Pyomo-Cookbook/notebooks/04.02-Machine-Bottleneck.html
57. python - Single machine scheduling - due dates constraints - Stack Overflow, accessed March 27, 2026, https://stackoverflow.com/questions/76682960/single-machine-scheduling-due-dates-constraints
58. Scheduling identical parallel machines to minimize total tardiness - ResearchGate, accessed March 27, 2026, https://www.researchgate.net/publication/223344786_Scheduling_identical_parallel_machines_to_minimize_total_tardiness
59. Project scheduling under uncertainty: Survey and research potentials. European Journal of Operational Research, 165(2), 289-306 | Request PDF - ResearchGate, accessed March 27, 2026, https://www.researchgate.net/publication/222817540_Project_scheduling_under_uncertainty_Survey_and_research_potentials_European_Journal_of_Operational_Research_1652_289-306
60. A Heuristic Method for a Single machine Scheduling Problem with Planned Maintenance - ScholarWorks, accessed March 27, 2026, https://scholarworks.calstate.edu/downloads/qf85nj096
61. AlgTUDelft/Decision-Diagrams-for-Single-Machine ... - GitHub, accessed March 27, 2026, https://github.com/AlgTUDelft/Decision-Diagrams-for-Single-Machine-Scheduling
62. A two-stage Ant Colony Optimization algorithm to minimize the makespan on

unrelated parallel machines with sequence-dependent setup times | Request PDF - ResearchGate, accessed March 27, 2026, https://www.researchgate.net/publication/226584445_A_two-stage_Ant_Colony_Optimization_algorithm_to_minimize_the_makespan_on_unrelated_parallel_machines_with_sequence-dependent_setup_times

63. A hybrid column generation-based heuristic for solving the parallel machine scheduling problem with sequence-dependent set-up times, accessed March 27, 2026, https://royalsocietypublishing.org/rsta/article/383/2310/20240380/364111/A-hybrid-column-generation-based-heuristic-for
64. Single machine scheduling problems with sequence-dependent setup times and precedence delays - PMC, accessed March 27, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC9177696/